HealthMate

Development Workload - Equal Split

6 November 2026 | Kotlin + Jetpack Compose + Room

Repository: github.com/navod718/healthmate

Scope: Development only: application setup, complete features, tests and integration. Report writing, demonstration video, publishing and submission tasks are excluded. Each person owns the UI, logic and database operations for their features.

1. Equal effort allocation

Each person is allocated 50 effort points, representing 50% of the estimated development workload. Points are planning estimates, not coursework marks or hours.

Development package	Person 1 - You	Person 2 - Your friend	Points each
Primary feature	Add Record	Edit Record	15
Second feature	Dashboard	Records History, date filtering and deletion	15
Shared foundation ownership	Theme, navigation and reusable Compose components	Room entity, DAO, database and repository foundation	8
Feature testing	Add, dashboard, validation and navigation checks	Edit, filter, delete and persistence checks	7
Integration and fixes	Connect owned features; review and fix integration issues	Connect owned features; review and fix integration issues	5
Total	50% of development effort	50% of development effort	50

2. Working principles

• Divide ownership by complete features. Both people write Compose UI, application logic and feature-specific data operations.

• The Room foundation owner sets up shared infrastructure; each feature owner implements the queries and repository/ViewModel behavior needed by their features.

• The UI foundation owner sets up shared styling and navigation; each feature owner builds their own screens and components.

• Use shared validation rules for Add and Edit. Agree who changes shared files before editing them.

• Review the balance after the first working feature. Reassign remaining fixes or integration tasks if one person has substantially more work.

3. Person 1 - You

Proposed branch: feature/add-record-dashboard

Add Record - 15 points

• Build the Compose form with date, steps, calories and water inputs.

• Connect input state and validation to a ViewModel; show useful field and save errors.

• Implement insertion through the repository and Room DAO.

• Clear or close the form after a successful save and refresh dependent screens.

Dashboard - 15 points

• Build today's date display and summary cards for steps, calories and water.

• Implement daily SUM queries and repository/ViewModel state for totals.

• React to added, edited and deleted records; show zero totals when no records exist.

• Refresh today's totals when the app resumes after the local date changes.

Foundation - 8 points

• Set up Compose Material 3 colours, typography and reusable cards, fields and buttons.

• Implement navigation between Dashboard, Records and Add/Edit destinations.

• Agree route arguments and edit-record navigation with Person 2.

Tests and integration - 12 points

• Test valid saves, invalid input, multiple entries per day, correct totals and navigation.

• Verify loading, empty and error states on owned screens.

• Integrate owned features, review Person 2's changes and fix agreed integration defects.

Done when: Valid records persist, invalid records are blocked, the dashboard shows accurate reactive totals, navigation works and owned checks pass on the integrated app.

4. Person 2 - Your friend

Proposed branch: feature/records-management

Edit Record - 15 points

• Build the Compose edit form and load a record by id.

• Pre-fill fields, handle a missing record and apply shared validation rules.

• Implement update operations through the ViewModel, repository and Room DAO.

• Preserve the record id and refresh history and dashboard after saving.

Records management - 15 points

• Build a dated record list ordered newest first, with loading and empty states.

• Implement the date picker/filter and matching DAO and repository queries.

• Connect record selection to edit navigation.

• Build delete confirmation and implement cancellation and confirmed deletion through Room.

Foundation - 8 points

• Set up the Room database, health_records entity, DAO and repository interfaces.

• Configure database dependencies and a shared database instance; coordinate build changes.

• Provide explicit demo-data loading with protection against accidental duplicate seeding.

Tests and integration - 12 points

• Test loading and updating existing records, date filtering, cancelled/confirmed deletion and persistence across restart.

• Verify empty results, missing-record handling and demo-data behavior.

• Integrate owned features, review Person 1's changes and fix agreed integration defects.

Done when: History and filters show correct records, edits and deletion persist, dependent totals refresh and owned checks pass on the integrated app.

5. Shared development agreement

Item	Agreement
Model	id: Long; date: YYYY-MM-DD text; steps, calories and water: Int. Water uses ml.
Validation	All fields required; non-negative whole numbers; at least one positive health value; no future dates. Share rules between Add and Edit.
Record behavior	Multiple entries may exist per day. Dashboard totals sum all matching records.
Architecture	Compose screen -> ViewModel -> Repository -> Room DAO -> SQLite. Use StateFlow for UI state and Flow for reactive reads.
Contracts	Agree repository method signatures, UI state classes, route arguments and error handling before feature development.
Shared files	Coordinate edits to Gradle configuration, navigation, validation, entity, DAO and repository interfaces.

6. Development sequence

• Together: agree the screens, record rules, package structure and interfaces.

• Create one minimal compilable Android project foundation and merge it into main. Both feature branches then update from main.

• Person 1 builds UI foundations and Add Record; Person 2 builds Room foundations and Edit Record. Use agreed sample state until shared dependencies are available.

• Person 1 builds Dashboard; Person 2 builds Records History, filtering and deletion.

• Integrate through reviewed pull requests into main, update both branches and resolve conflicts.

• Together: verify add -> view -> edit -> filter -> delete, check dashboard totals after each mutation and restart the app to verify persistence.

7. Branch and completion notes

The proposed feature branches above describe this allocation. Existing branches feature/compose-ui and feature/room-data reflect the earlier split; this PDF does not rename them. Use only the Navod718/healthmate repository and meaningful incremental commits.

Development is complete when the integrated app builds, all required features work together and actual test results support acceptance. Effort points may be adjusted jointly while maintaining a 50/50 overall target.

Coursework context: The supplied brief identifies individual coursework. Follow the lecturer's rules for collaboration and acknowledgement of help. This document is a development allocation, not evidence that joint submission is permitted.